Name: Niyonkuru shalom
RegNo: 224011591

Object Oriented Programming

Pre-CAT of OOP C++

Q1) A business application requires a Product class to manage inventory.

Code:

question1.cpp > main()

```cpp
#include <iostream>
using namespace std;

class Product {
private:
    // Requirement 1: Data members are hidden (private)
    int productID;
    double price;

public:
    // Requirement 2: Constructor with validation and member initializer list
    Product(int id, double initialPrice) : productID(id) {
        if (initialPrice < 0.0) {
            cout << "Notice: Initial price cannot be negative. Setting to 0.0." << endl;
            price = 0.0;
        } else {
            price = initialPrice;
        }
    }

    // Requirement 3: Public method to update price with validation
    void setPrice(double newPrice) {
        if (newPrice >= 0.0) {
            price = newPrice;
            cout << "Success: Price updated to RFW " <<price << endl;
        } else {
            cout << "Error: Update rejected: New price (RFW " << newPrice << ") cannot be negati
        }
    }

    // Helper method to display product details
    void display() const {
        cout << "Product ID: " << productID
                << " | Price: RFW " <<price << endl;
    }
};

int main() {
    cout << "==========================================" << endl;
    cout << "            QUESTION ONE SOLUTION          " << endl;
    cout << "==========================================" << endl;

    // Normal initialization
    cout << "Initializing Product(101, 25.50)" << endl;
    Product p1(101, 25.50);
    p1.display();
    cout << endl;

    // Negative initial price
    cout << "Initializing Product(102, -10.00)" << endl;
    Product p2(102, -10.00);
    p2.display();
    cout << endl;

    // Validating setPrice updates
    cout << "Updating price to 30.00" << endl;
    p1.setPrice(30.00);
    p1.display();
    cout << endl;

    // Validating setPrice rejection
    cout << "Attempting to set negative price (-5.00)" << endl;
    p1.setPrice(-5.00);
    p1.display();
    cout << endl;

    return 0;
}
```

Output:

```
Problems   Output   Debug Console   Terminal   Ports

PS D:\cpp\Shalom-cat-cpp> .\question1
==========================================
            QUESTION ONE SOLUTION
==========================================
Initializing Product(101, 25.50)
Product ID: 101 | Price: RFW 25.5

Initializing Product(102, -10.00)
Notice: Initial price cannot be negative. Setting to 0.0.
Product ID: 102 | Price: RFW 0

Updating price to 30.00
Success: Price updated to RFW 30
Product ID: 101 | Price: RFW 30

Attempting to set negative price (-5.00)
Error: Update rejected: New price (RFW -5) cannot be negative.
Product ID: 101 | Price: RFW 30

PS D:\cpp\Shalom-cat-cpp>
```

Q2) The following class manages a dynamic description for a business report.

Code:

question2.cpp > ...

```cpp
#include <iostream>
#include <cstring>
using namespace std;

class BusinessReport {
private:
    char* description;

public:
    // Original Constructor Provided in the Assignment
    BusinessReport(const char* desc) {
        if (desc) {
            description = new char[strlen(desc) + 1];
            strcpy(description, desc);
        } else {
            description = nullptr;
        }
    }

    // Implement Destructor (RAII Principle)
    ~BusinessReport() {
        delete[] description;
        cout << "Destructor: Dynamic memory deleted safely." << endl;
    }

    // Implement Copy Constructor (Deep Copy)
    BusinessReport(const BusinessReport& other) {
        if (other.description) {
            description = new char[strlen(other.description) + 1];
            strcpy(description, other.description);
        } else {
            description = nullptr;
        }
        cout << "Copy Constructor: Performed deep copy of description." << endl;
    }

    BusinessReport& operator=(const BusinessReport& other) {
        if (this != &other) {
            delete[] description;
            if (other.description) {
                description = new char[strlen(other.description) + 1];
                strcpy(description, other.description);
            } else {
                description = nullptr;
            }
            cout << "Assignment Operator: Deep copy completed after cleanup." << endl;
        }
        return *this;
    }

    // Helper method to display report status
    void display() const {
        if (description) {
            cout << "Report Description: " << description << endl;
        } else {
            cout << "Report Description: [Empty]" << endl;
        }
    }
};
```

```
};

int main() {
    cout << "==========================================" << endl;
    cout << "            QUESTION TWO SOLUTION          " << endl;
    cout << "==========================================" << endl;

    // Initial creation and display
    cout << "Initializing 'reportA'" << endl;
    BusinessReport reportA("Annual Financial Audit 2026");
    reportA.display();
    cout << endl;

    // Deep Copy via Copy Constructor
    cout << "Creating 'reportB' as a copy of 'reportA'" << endl;
    {
        BusinessReport reportB = reportA; // Calls Copy Constructor
        reportB.display();
        cout << "Notice: 'reportB' is about to go out of scope..." << endl;
    }
    cout << "Notice: 'reportB' has been destroyed." << endl << endl;

    // Verifying 'reportA' is still valid (No shallow copy errors)
    cout << "Verifying 'reportA' remains valid" << endl;
    reportA.display();
    cout << "Success: 'reportA' was not affected by 'reportB' destruction." << endl << endl;

    // Deep Copy via Assignment Operator
    cout << "Testing Assignment Operator ('reportC = reportA')" << endl;
    BusinessReport reportC("Placeholder");
    reportC = reportA;
    reportC.display();
    cout << "Success: Assignment successful." << endl << endl;

    cout << "End of Test Program. 'reportA' and 'reportC' will be cleaned up." << endl;
    return 0;
}
```

Output:

```
PS D:\cpp\Shalom-cat-cpp> .\question2
==========================================
            QUESTION TWO SOLUTION
==========================================
Initializing 'reportA'
Report Description: Annual Financial Audit 2026

Creating 'reportB' as a copy of 'reportA'
Copy Constructor: Performed deep copy of description.
Report Description: Annual Financial Audit 2026
Notice: 'reportB' is about to go out of scope...
Destructor: Dynamic memory deleted safely.
Notice: 'reportB' has been destroyed.

Verifying 'reportA' remains valid
Report Description: Annual Financial Audit 2026
Success: 'reportA' was not affected by 'reportB' destruction.

Testing Assignment Operator ('reportC = reportA')
Assignment Operator: Deep copy completed after cleanup.
Report Description: Annual Financial Audit 2026
Success: Assignment successful.

End of Test Program. 'reportA' and 'reportC' will be cleaned up.
Destructor: Dynamic memory deleted safely.
Destructor: Dynamic memory deleted safely.
PS D:\cpp\Shalom-cat-cpp>
```

Q3) inancial tracking system.

Code:

question3.cpp > ...

```cpp
#include <iostream>

class TransactionCounter {
private:
    int count;

public:
    // Initialize count to 0 using member initializer list (Day 1/2 style)
    TransactionCounter() : count(0) {}

    /**
     * Implement prefix ++ operator (Day 4.5 style)
     *
     * Why is returning a reference (TransactionCounter&) preferred?
     * 1. Efficiency: As per Lecture 4.5, avoiding a copy saves memory and processing time.
     * 2. Chaining: It allows multiple operations like '++++obj' (Lecture 4.5).
     * 3. Consistency: It mimics the behaviour of built-in integer types.
     */
    TransactionCounter& operator++() {
        ++count;
        return *this;
    }

    // Helper method to display count
    void display() const {
        std::cout << "Current Sales Count: " << count << std::endl;
    }
};

int main() {
    std::cout << "==========================================" << std::endl;
    std::cout << "          QUESTION THREE SOLUTION          " << std::endl;
    std::cout << "==========================================" << std::endl;
    std::cout << "System: Financial Transaction Tracking" << std::endl;
    std::cout << "Feature: Prefix Operator Overloading" << std::endl;
    std::cout << "------------------------------------------" << std::endl << std::endl;

    // Initialization
    std::cout << "Initializing TransactionCounter object" << std::endl;
    TransactionCounter sales;
    sales.display();
    std::cout << std::endl;


    std::cout << "Applying prefix increment (++sales)" << std::endl;
    ++sales;
    sales.display();
    std::cout << "Success: Sales count incremented." << std::endl << std::endl;

    std::cout << "Applying chained prefix increment (++++sales)" << std::endl;
    std::cout << "Chained operations work because the operator returns a reference to *this." <
    ++++sales;
    sales.display();
    std::cout << "Success: Chained increment verified (Value: 3)." << std::endl << std::endl;

    return 0;
}
```

Output:

```
Problems   Output   Debug Console   Terminal   Ports                    powershell

PS D:\cpp\Shalom-cat-cpp> g++ question3.cpp -o question3
PS D:\cpp\Shalom-cat-cpp> .\question3
==========================================
          QUESTION THREE SOLUTION
==========================================
System: Financial Transaction Tracking
Feature: Prefix Operator Overloading
------------------------------------------

Initializing TransactionCounter object
Current Sales Count: 0

Applying prefix increment (++sales)
Current Sales Count: 1
Success: Sales count incremented.

Applying chained prefix increment (++++sales)
Chained operations work because the operator returns a reference to *this.
Current Sales Count: 3
Success: Chained increment verified (Value: 3).

PS D:\cpp\Shalom-cat-cpp>
```

Explaination:why returning a reference (TransactionCounter&) is preferred for prefix increment?

Answer: Implement prefix ++ operator (Day 4.5 style)

1. Efficiency: As our Lecture note day 4.5 referenced, it is because, avoiding a copy saves memory and processing time.
2. Chaining: It allows multiple operations like '++++obj'.
3. Consistency: It mimics the behaviour of built-in integer types.

Q4)

EXPLANATION: Purpose of Header Guards (Lecture note day 2)

- Header guards are used to prevent multiple inclusions of the same header file
- within a single translation unit. If a header is included multiple times (due to
- complex inheritance or nested includes), without guards, the compiler would
- see duplicate definitions of the same class, leading to 'redefinition' errors.

Mechanism:

- **#ifndef ACCOUNT_H**: Checks if the unique identifiers 'ACCOUNT_H' has not been defined.
- **#define ACCOUNT_H**: Defines the name(identifiers) so future checks fail.
- **#endif**: Marks the end of the guarded content.

Code: Account.h

```
Account.h > ...
#ifndef ACCOUNT_H
#define ACCOUNT_H


class Account {
private:
    double balance;

public:
    // Constructor
    Account(double initialBalance = 0.0);

    // Mutator
    void deposit(double amt);

    // Accessor
    double getBalance() const;
};

#endif

```

Account.cpp

Account.cpp

```cpp
#include "Account.h"
#include <iostream>

// Constructor: Initializes balance with validation
Account::Account(double initialBalance) {
    if (initialBalance >= 0.0) {
        balance = initialBalance;
    } else {
        balance = 0.0;
        std::cout << "Notice: Initial balance cannot be negative. Setting to 0.0.
    }
}

// Mutator: Validates deposit amount
void Account::deposit(double amount) {
    if (amount > 0.0) {
        balance += amount;
    } else {
        std::cout << "Error: Deposit amount must be positive." << std::endl;
    }
}

// Accessor: Returns current balance
double Account::getBalance() const {
    return balance;
}
```

Main.cpp

main.cpp

```cpp
#include <iostream>
#include "Account.h"
using namespace std;

int main() {
    cout << "=========================================" << endl;
    cout << "           QUESTION FOUR SOLUTION          " << endl;
    cout << "=========================================" << endl;

    // Creation
    cout << "Creating account with initial balance of RFW 500.25" << endl;
    Account myAcc(500.25);
    cout << "Current Balance: RFW " << myAcc.getBalance() << endl;
    cout << endl;

    // Deposit
    cout << "Depositing RFW 150.75" << endl;
    myAcc.deposit(150.75);
    cout << "[Success] Updated Balance: RFW " << myAcc.getBalance() << endl;
    cout << endl;

    // Validation (Negative Deposit)
    cout << "Attempting to deposit a negative amount (-RFW 50.00)" << endl;
    myAcc.deposit(-50.00);
    cout << "[Notice] Deposit rejected. Balance remains: RFW "<< myAcc.getBalance
    cout << endl;

    cout << "Verification Complete: Class structure and Header Guards confirmed."
    return 0;
}
```

Output:

Problems Output Debug Console Terminal Ports powershell

```
PS D:\cpp\Shalom-cat-cpp> g++ Account.cpp main.cpp -o question4
PS D:\cpp\Shalom-cat-cpp> .\question4
=========================================
           QUESTION FOUR SOLUTION
=========================================
Creating account with initial balance of RFW 500.25
Current Balance: RFW 500.25

Depositing RFW 150.75
Success: Updated Balance: RFW 651

Attempting to deposit a negative amount (-RFW 50.00)
Error: Deposit amount must be positive.
Notice: Deposit rejected. Balance remains: RFW 651

Verification Complete: Class structure and Header Guards confirmed.
PS D:\cpp\Shalom-cat-cpp>
```